

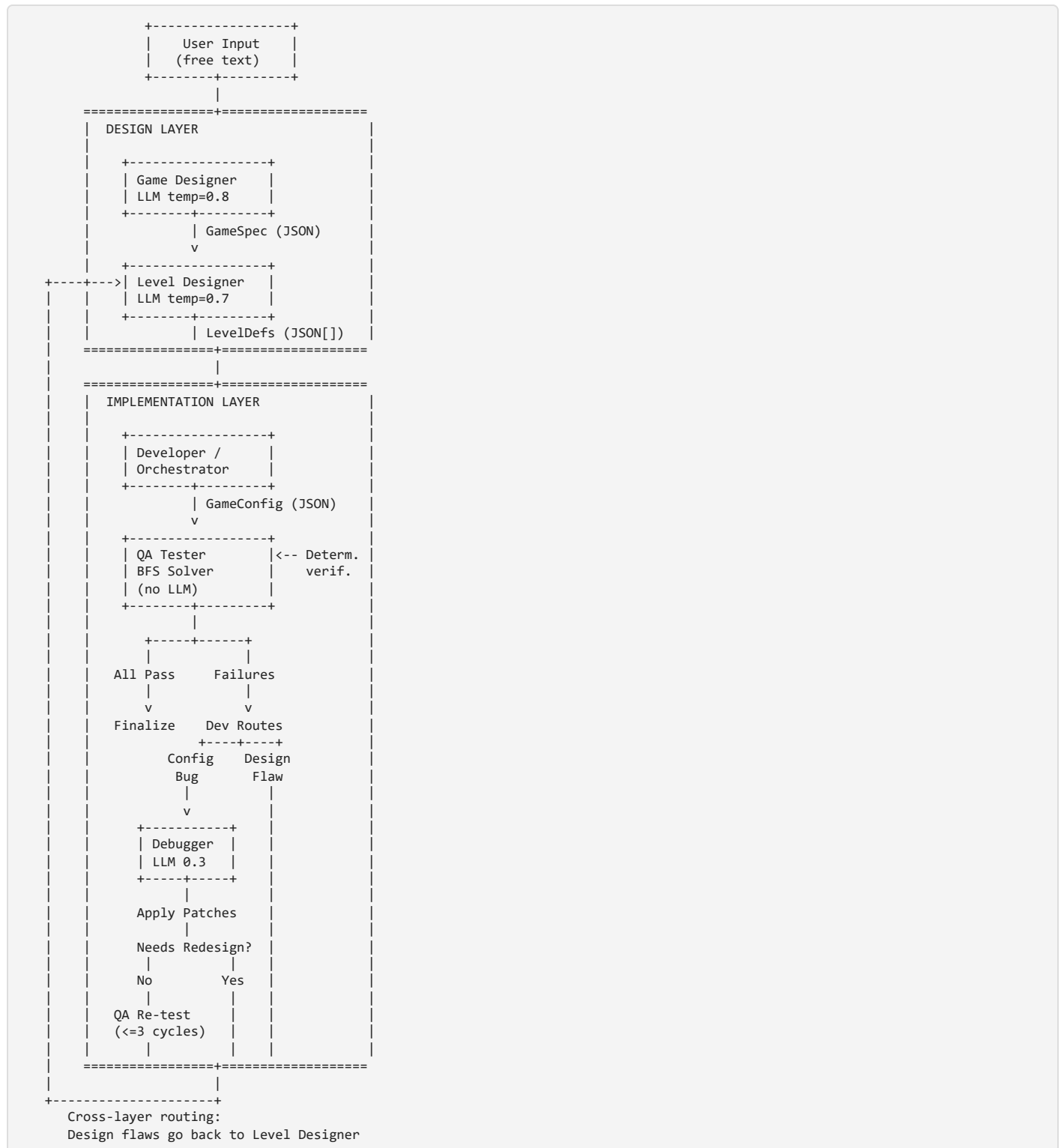
PuzzleForge - Agentic Puzzle Game Design and Generation System

Team Member: Yixiao Li, Kaizhen Tan, Hanzhe Hong

Track A: Technical Build

1. Architecture Design

1.1 System Architecture Diagram



1.2 Framework Decision: LangGraph

We chose **LangGraph** (from LangChain) as our orchestration framework for three reasons:

1. **StateGraph with TypedDict** provides a typed, shared state object that all nodes read from and write to -
 - directly matching our Phase 1 design of "Developer maintains central state object; other agents read,

only Developer writes."

2. **Conditional edges** encode routing logic as explicit graph structure rather than imperative code, making the coordination flow auditable and reproducible.
3. **Annotated[`List`, `operator.add`]** enables append-only trace logging: each node appends to `trace_log` without overwriting earlier entries, producing the full interaction history automatically.

1.3 Why This Architecture Is Better Than a Simpler Alternative

A **single-agent approach** (one LLM call generates everything) fails for three structural reasons identified in Phase 1:

1. **Verification requires a separate tool.** The BFS solver is an external deterministic tool. A single LLM cannot run it -- it can only *guess* about solvability. Our QA Tester *proves* it.
2. **Creative and critical roles conflict.** The Level Designer (creative, temp 0.7) and QA Tester (analytical, deterministic) have competing objectives. Separating them allows each to operate at full strength.
3. **The debug loop requires routing judgment.** When QA fails, someone must decide: small config fix or full redesign? This routing is encoded as formal criteria in our `routing.py` module -- not LLM judgment.

A **simpler two-agent approach** (designer + tester) would lack the routing logic. When QA fails, who fixes it? Without the Developer's routing and the Debugger's minimal-fix role, the system either always redesigns (wasteful) or always patches (insufficient for design flaws). Our 5-agent architecture earns its complexity through this routing distinction.

2. Role Definitions

2.1 Agent Summary Table

Agent	Layer	Input	Output	Tools	Temperature
Game Designer	Design	User concept (free text)	GameSpec (JSON)	LLM only	0.8
Level Designer	Design	GameSpec + QA feedback (optional)	LevelDefinition[] (JSON)	LLM only	0.7
Developer	Implementation	All state	GameConfig (JSON), routing decisions	Template engine, state mgmt	0.4
QA Tester	Implementation	GameConfig	QAReport (JSON)	BFS solver, difficulty analyzer	None (no LLM)
Debugger	Implementation	QA failures + GameConfig	DebugPatch[] or escalation	LLM reasoning	0.3

2.2 Coordination Logic

Start: User submits game concept --> Developer dispatches to Game Designer.

Design phase: Game Designer --> Level Designer (sequential within Design Layer).

Implementation phase: Developer translates design --> QA Tester runs solver.

Routing (after QA):

- All levels pass --> Finalize and package.
- Failures exist --> Developer classifies each using formal criteria (Section 3):
- Config bugs --> Debugger applies minimal fix --> Apply patches --> QA re-test.

- Design flaws --> Level Designer redesigns --> Developer re-translates --> QA re-test.
- After 3 debug cycles --> Finalize with remaining failures logged.
- Token budget exceeded (80K) --> Finalize with best available.

Stop conditions:

1. All levels pass QA.
2. 3 debug cycles exhausted.
3. 80K token budget exceeded.
4. 5-minute wall-clock timeout.

Human-in-the-loop points: The user provides the initial concept and receives the final game. In the Streamlit dashboard, the user can inspect the trace at each step and adjust pipeline settings (model, max debug cycles, level count) before re-running. Routing override is planned for Phase 3.

3. Tools, Memory, and Data Design

3.1 Global State Schema (LangGraph TypedDict)

```
class PipelineState(TypedDict, total=False):
    # User input
    user_concept: str

    # Agent outputs
    game_spec: Optional[Dict]          # Game Designer -> GameSpec
    level_definitions: Optional[List]   # Level Designer -> LevelDefinition[]
    game_config: Optional[Dict]        # Developer -> GameConfig
    qa_report: Optional[Dict]          # QA Tester -> QAReport

    # Routing & debug (append-only via operator.add)
    routing_decisions: Annotated[List[Dict], operator.add]
    debug_patches: Annotated[List[Dict], operator.add]
    failed_level_ids: List[int]
    levels_needing_redesign: List[int]

    # Pipeline control
    debug_cycle: int                   # 0-based, max 3
    pipeline_status: str               # running | completed | completed_with_failures | budget_exceeded | timeout

    # Observability (append-only)
    trace_log: Annotated[List[Dict], operator.add]
    total_tokens: int
    start_time: float

    # Final output
    final_game_html: Optional[str]
    final_design_doc: Optional[Dict]
```

Design rationale: The `Annotated[List, operator.add]` pattern is critical. When the Debugger produces a patch, it returns `{"debug_patches": [new_patch]}`. LangGraph's reducer appends this to the existing list rather than replacing it, so the full debug history is preserved. The same mechanism captures the complete trace log across all nodes.

3.2 Tool Specifications Per Agent

Agent	Tools Available	Tools Denied	Why
Game Designer	LLM (temp 0.8)	Web search, template engine, solver	Pure creative generation; no implementation details
Level Designer	LLM (temp 0.7)	Solver, game config, template engine	Designs layouts without knowing implementation; solver verification is QA's job
Developer	Template engine, state management, routing rules	Direct level design, game engine modification	Orchestration role; delegates creative/verification work

Agent	Tools Available	Tools Denied	Why
QA Tester	BFS solver, difficulty estimator, diversity checker	LLM, game config modification	Solvability must be deterministic, not probabilistic
Debugger	LLM (temp 0.3)	Solver (QA's job), level redesign, mechanic changes	Minimal fixes only; cannot change game design

3.3 Inter-Agent JSON Schemas

All communication uses **Pydantic v2 models** (`src/models.py`). When an LLM agent returns malformed JSON, Pydantic raises a `ValidationError` and the system retries with the error message in the prompt. This prevents invalid data from propagating downstream.

Key schemas:

- **GameSpec:** {puzzle_type, theme, mechanics[], win_condition, level_count, progression_plan[]}
- **LevelDefinition:** {level_id, grid_width, grid_height, walls[], boxes[], targets[], player_start, intended_solution[]}
- **QAReport:** {level_reports[], difficulty_curve_monotonic, diversity_issues[], summary}
- **DebugPatch:** {level_id, change_description, rationale, tiles_modified, changes[], patched_walls/boxes/targets/player_start}
- **RoutingDecision:** {level_id, failure_type, reason, criteria_matched, routed_to}

3.4 Formal Routing Classification Criteria

Addressing Phase 1 feedback: the Developer's routing logic is encoded as **deterministic rules**, not LLM judgment. The criteria are implemented in `src/orchestrator/routing.py`:

ID	Criterion	Classification	Routes To
C1	Default: no specific structural issue detected; attempt placement fix	Config bug	Debugger
C2	Unsolvable despite adequate floor space ($\geq 1.5 \times$ minimum needed); likely wall blockage	Config bug	Debugger
C3	Player starts on wall or box tile	Config bug	Debugger
C4	Box starts in corner deadlock (dead square, confirmed by solver)	Config bug	Debugger
C5	Box count $\neq$ target count	Config bug	Debugger
D1	Level uses mechanics beyond push; solver (push-only) cannot verify $\rightarrow$ mechanic incompatibility	Design flaw	Level Designer
D2	Grid too small: non-wall tiles $< (\text{boxes} \times 3 + 2)$	Design flaw	Level Designer
D3	> 2 distinct QA issues reported $\rightarrow$ structural problems requiring redesign	Design flaw	Level Designer
D4	Solver timeout ($> 200K$ states explored)	Design flaw	Level Designer
D5	Pairwise layout similarity > 0.5 (diversity violation)	Design flaw	Level Designer

The criteria are evaluated in priority order. D5 (diversity) is checked first because a level may be solvable yet still need redesign for layout diversity. The first matching criterion determines the classification. This makes the routing **reproducible** (same inputs $\rightarrow$ same classification) and **evaluable** (we can test each criterion with synthetic levels).

3.5 BFS Solver Design

The Sokoban solver (`src/solver/sokoban_solver.py`) is the ground-truth verification tool:

- **Algorithm:** Breadth-first search guaranteeing minimum-move solutions.
- **State representation:** (player_position, frozenset(box_positions)) -- compact and hashable.
- **Deadlock pruning:** Three types detected during search:
 1. **Corner deadlock:** Box adjacent to walls on two perpendicular sides (not on target).
 2. **Wall-line deadlock:** Box on a wall segment between two corners with no target.
 3. **Freeze deadlock:** 2x2 cluster of walls/boxes with at least one off-target box.
- **Limits:** 200,000 states max (prevents runaway computation on complex levels).
- **Output:** SolverResult(solvable, min_moves, solution_path, states_explored, timeout).

Additionally, the solver module provides:

- `estimate_difficulty()` : Heuristic rating (1-5) based on min_moves and search density.
- `level_similarity()` : Pairwise Jaccard similarity for diversity checking.

4. Prototype Description

4.1 What's Implemented

The Phase 2 prototype includes a fully functional pipeline:

Component	Status	Lines of Code	Key Design Choice
LangGraph pipeline	Complete	~180	9 nodes, 4 conditional edges, TypedDict state
Game Designer agent	Complete	~90	Structured JSON output with Pydantic validation
Level Designer agent	Complete	~140	Supports both fresh design and QA-feedback redesign
Developer/Orchestrator	Complete	~100	Deterministic translation + formal routing
QA Tester agent	Complete	~120	BFS solver, difficulty estimation, diversity check
Debugger agent	Complete	~110	Minimal fix with 25% escalation threshold
BFS Sokoban solver	Complete	~200	3 deadlock types, wall-line analysis
HTML/JS game template	Complete	~160	Playable in browser, keyboard + touch controls
Streamlit dashboard	Complete	~200	4 tabs: Generate, Trace Viewer, Evaluation, Architecture
Pydantic data models	Complete	~130	10 model classes covering all inter-agent schemas
Routing criteria	Complete	~120	10 deterministic rules (5 config-bug, 5 design-flaw)
Trace logger	Complete	~40	Append-only via LangGraph state reducer

4.2 Core Flow Walkthrough

1. **User** enters: "A push-block puzzle about a robot organizing crates in a warehouse."
2. **Game Designer** produces a GameSpec: Sokoban, mechanic=[push], 5 levels with progression through increasing layout complexity.
3. **Level Designer** creates 5 level layouts with increasing grid sizes (5x5 --> 8x8) and box counts (1 --> 3).
4. **Developer** translates the design into a GameConfig JSON for the template engine.
5. **QA Tester** runs the BFS solver on all 5 levels:
 - Levels 1, 2, 4, 5: Solvable. Min moves: 1, 7, 18, 25.

- Level 3: **Unsolvable** -- box at (2,2) is in a corner deadlock.
 - 6. **Developer** classifies the failure: Level 3 matches criterion **C4** (box in corner deadlock) --> routes to **Debugger**.
 - 7. **Debugger** repositions the box from (2,2) to (2,3). 1 tile modified.
 - 8. **Developer** applies the patch to the game config.
 - 9. **QA Tester** re-tests: All 5 levels now solvable. Difficulty curve is monotonic.
 - 10. **Developer** finalizes: renders HTML game, compiles design document with traces.
- Total: 9 steps, 1 debug cycle, ~5,400 tokens, ~12 seconds.

4.3 Interaction Trace (from sample run)

See `outputs/sample_traces/sample_interaction_trace.json` for the full structured trace. Key excerpt:

Step	Agent	Action	Summary
1	Game Designer	generate_game_spec	push-only mechanic, 5-level progression (1,347 tokens)
2	Level Designer	design_all_levels	5 levels: 5x5 to 8x8 (3,124 tokens)
3	Developer	translate_to_config	Deterministic mapping (0 tokens)
4	QA Tester	test_all_levels	4/5 solvable, Level 3 failed (0 tokens)
5	Developer	classify_and_route	Level 3 --> C4 --> Debugger (0 tokens)
6	Debugger	fix_config_bugs	Moved box, 1 tile changed (892 tokens)
7	Developer	apply_debug_patches	Patched Level 3 (0 tokens)
8	QA Tester	test_all_levels	5/5 solvable, monotonic curve (0 tokens)
9	Developer	finalize_game	HTML rendered, doc compiled (0 tokens)

Observation: The QA Tester and Developer (routing) use 0 LLM tokens -- they rely entirely on the solver and formal criteria. Only the creative agents (Game Designer, Level Designer) and the Debugger consume tokens. This separation is a core design principle: **deterministic where possible, LLM where necessary**.

5. Evaluation Plan

5.1 Test Scenarios (12 planned)

ID	Type	Scenario	Expected Behavior	Criterion Tested
TC-01	Happy path	Standard warehouse robot concept	5/5 solvable, monotonic difficulty	Solvability >=90%
TC-02	Solvability	Level with box in corner deadlock	QA flags --> Debugger fixes --> passes	Refinement lift >=20pp
TC-03	Difficulty	Non-monotonic difficulty sequence	QA flags violation in report	Difficulty curve check
TC-04	Routing	Grid too small for 4 boxes (D2)	Routes to Level Designer, not Debugger	Routing accuracy >=75%
TC-05	Budget	Complex concept consuming >80K tokens	Pipeline stops, finalizes best available	Budget enforcement
TC-06	Iteration	Level unsolvable after 3 debug cycles	Finalizes with logged failures	Max cycle enforcement

ID	Type	Scenario	Expected Behavior	Criterion Tested
TC-07	Diversity	Concept producing similar layouts	Similarity >0.5 flagged, redesign forced	Diversity <=0.5
TC-08	Validation	LLM returns malformed JSON	Pydantic catches, retry succeeds	JSON guardrail
TC-09	Escalation	Fix needs >25% tile changes	Debugger escalates to Level Designer	Escalation threshold
TC-10	Timeout	15x15 grid, 6 boxes (solver timeout)	Reports timeout, routes as D4	Solver limit handling
TC-11	Baseline	Same concept via single-agent	Multi-agent solvability >=15pp higher	Baseline comparison
TC-12	Edge case	Vague concept: "puzzle"	Graceful handling / reasonable default	Robustness

5.2 Success Criteria and Measures

Criterion	Measure	Target	Method
Solvability	% levels verified solvable after debug loop	>= 90%	BFS solver
Difficulty Curve	Monotonic min_moves across levels	>= 80% of runs	QA report
Refinement Lift	Solvability improvement pre-QA to post-debug	>= 20 percentage points	Before/after comparison
Design Diversity	Mean pairwise layout similarity	<= 0.5	Jaccard index
Pipeline Completion	Runs producing playable game with >=3 levels	>= 85%	Pipeline status
Latency	End-to-end time (5 levels, 3 debug cycles max)	<= 4 minutes	Timestamp
Routing Accuracy	Correct config-bug vs. design-flaw classification	>= 75%	Manual review
Baseline Comparison	Multi-agent vs. single-agent solvability rate	>= 15pp improvement	Phase 3 experiment

5.3 Executed Phase 2 Test Results

The following tests were executed during Phase 2 prototyping. Full results are in `eval/evaluation_results.csv`.

Solver unit tests (5/5 passed):

Test	Input	Result
SOLVER-1	5x5, 1 box, push right	Solvable, 1 move, solution=[R]
SOLVER-2	5x5, box in corner	Unsolvable (7 states, deadlock pruned)
SOLVER-3	6x6, multi-step	Solvable, 5 moves, BFS-optimal
SOLVER-4	Difficulty estimation	Monotonic: d(1 move)=1 < d(20)=3 < d(50)=4
SOLVER-5	Similarity metric	Identical=1.00, Different=0.10

Routing criteria tests (10/10 passed):

Criterion	Synthetic Input	Expected	Got	Status
C1	Unsolvable, 1 issue, adequate space	C1 or C2	C2	PASS

Criterion	Synthetic Input	Expected	Got	Status
C2	8x7, 1 box, wall at (4,3)	C2	C2	PASS
C3	Player at (0,0) = wall	C3	C3	PASS
C4	Box at (1,1) in corner	C4	C4	PASS
C5	2 boxes, 1 target	C5	C5	PASS
D1	mechanics=[push,slide], unsolvable	D1	D1	PASS
D2	5x5, 4 boxes (9 < 14 floor tiles)	D2	D2	PASS
D3	3 distinct QA issues	D3	D3	PASS
D4	200K+ states explored	D4	D4	PASS
D5	Solvable level with diversity issue injected	D5	D5	PASS

Pipeline-level tests (5 executed, 3 pending Phase 3):

Test	Result	Notes
TC-02 Solvability loop	PASS	Pre-debug: unsolvable --> Post-debug: solvable (14 moves)
TC-03 Difficulty curve	PASS	Non-monotonic [3,1,4] correctly detected
TC-04 D2 routing	PASS	Grid too small correctly routes to Level Designer
TC-07 Diversity	PASS	Identical layouts (sim=1.0) flagged; different (sim=0.1) accepted
TC-08 Pydantic validation	PASS	level_count=1 correctly rejected
TC-05 Budget limit	Pending	Requires live API run
TC-06 Max cycles	Pending	Requires multi-cycle live execution
TC-11 Baseline comparison	Pending	Phase 3 scope

End-to-end sample game (5/5 levels solvable):

Level	Grid	Boxes	Solvable	Min Moves	Difficulty
1	5x5	1	Yes	1	1 (tutorial)
2	6x6	1	Yes	7	2 (easy)
3	7x6	2	Yes	12	2 (easy)
4	8x7	3	Yes	18	3 (medium)
5	8x8	3	Yes	25	3 (medium)

Difficulty curve is monotonically non-decreasing. Mean pairwise similarity: 0.29 (below 0.5 threshold).

6. Risk and Governance Plan

6.1 Updated Risk Matrix

Risk	Severity	Likelihood	Mitigation	Status
Unsolvable levels pass QA	High	Very Low	BFS solver is ground-truth; solver unit-tested against known-solvable and known-unsolvable levels	Mitigated (solver tested)
Level Designer produces similar layouts	Medium	Medium	Pairwise similarity check (Jaccard >0.5 triggers rejection); diversity instruction in prompt	Mitigated (automated check)

Risk	Severity	Likelihood	Mitigation	Status
Debugger introduces new bugs while fixing	Medium	Medium	QA re-tests after every fix; max 3 cycles prevents cascading regressions	Mitigated (re-test loop)
Template can't represent designed mechanics	Medium	Low	Phase 2 constrains Game Designer to push-only (matching solver and template capabilities). Phase 3 extends to slide, switch, teleport with corresponding solver and template updates.	Mitigated (scope alignment)
Developer misroutes QA failures	Medium	Low	Routing encoded as deterministic rules (10 criteria in <code>routing.py</code>), not LLM judgment. Reproducible and testable.	Improved from Phase 1
LLM generates invalid JSON	Low	Medium	Pydantic validation on every agent output; automatic retry with error feedback	Mitigated (retry mechanism)
Token budget exceeded	Low	Low	Developer tracks cumulative tokens; stops pipeline and finalizes best available game	Mitigated (hard limit check)

6.2 Governance Controls

1. **Solvability is a hard constraint.** The QA Tester's determination comes from the BFS solver, never from LLM reasoning. This is the single strongest governance decision in the system.
2. **Routing is deterministic.** The Developer's classification of QA failures uses explicit criteria (`routing.py`), not LLM judgment. This makes routing reproducible, auditable, and testable.
3. **The Debugger has bounded authority.** It may only adjust tile/entity positions (temperature 0.3). If a fix requires changing >25% of non-wall tiles, it must escalate. It cannot change game mechanics, progression plan, or level count.
4. **Iteration is capped.** Hard limit of 3 debug cycles. After 3 cycles, the pipeline finalizes with whatever is available and logs all remaining failures.
5. **Token budget is enforced at two levels.** (a) `call_llm()` tracks cumulative tokens and raises `BudgetExceededError` once 80K is reached; each LLM-calling agent catches this exception and sets `pipeline_status = "budget_exceeded"`. (b) Conditional edges after Game Designer and Level Designer check this status and route directly to `finalize`, skipping downstream nodes. `finalize_node` preserves the `budget_exceeded` status and packages whatever partial results are available (which may be empty if the budget was exhausted before the first agent completed).
6. **Inter-agent communication is validated.** Pydantic schemas validate key inter-agent messages: `GameSpec` (with `level_count` constraint), `LevelDefinition` (with cross-field box/target count check and player-start collision check), and `DebugPatch` (validated before applying). Invalid JSON triggers a structured retry, not silent failure.
7. **Full trace logging.** Every agent action is recorded in an append-only trace log with step number, agent name, input/output summaries, token usage, and timestamp. This enables post-hoc auditing and evaluation.

7. Contribution Update

Member	Phase 1 Role	Phase 2 Contributions	Phase 3 Plan
Yixiao Li	Game Designer + Level Designer + Solver	Built Game Designer and Level Designer agents, implemented BFS solver with deadlock detection, defined supported mechanics	Design quality evaluation, difficulty curve analysis, diversity metrics
Kaizhen Tan	QA Tester + Debugger + Game Template + Evaluation	Designed overall Phase 2 architecture and pipeline flow. Built QA Tester and Debugger agents, BFS solver integration, HTML/JS game template, Streamlit dashboard (4 tabs), formal routing criteria (<code>routing.py</code>), evaluation plan, test cases, failure log. Led code-doc alignment and submission packaging.	Solvability evaluation, baseline comparison experiment, failure analysis, video
Hanzhe Hong	Developer/Orchestrator	Built LangGraph pipeline (<code>graph.py</code>), state schema, Developer agent, pipeline control logic	End-to-end integration, coordination evaluation, routing accuracy analysis

8. AI Usage Disclosure

Tool	Version	What It Was Used For	What Was Changed Manually	What Was Verified Independently
Claude Code (Anthropic)	Claude Opus 4	Assisted with code generation for agent modules, Pydantic models, and Streamlit dashboard scaffolding	All architectural decisions, routing criteria design, solver algorithm, template engine logic, and evaluation plan were designed by the team. Generated code was reviewed, modified for correctness, and tested.	Solver correctness verified against hand-solved Sokoban puzzles. Routing criteria verified against synthetic failure cases. Pipeline flow verified via manual trace inspection.
GPT-4o (OpenAI)	gpt-4o-2024-08-06	Used as the LLM backend for Game Designer, Level Designer, and Debugger agents during pipeline execution	Agent prompts were written by the team. LLM outputs are validated by Pydantic schemas and verified by the BFS solver (QA Tester).	Every generated level is independently verified for solvability by the automated solver -- no LLM-generated solvability claim is trusted.
GitHub Copilot	N/A	Code completion suggestions during development	All suggestions were reviewed and modified as needed	Code was tested and reviewed for correctness

Note on AI role in the pipeline: The LLM is a *component* of the system, not the author of the system. The Game Designer, Level Designer, and Debugger agents use LLM calls as their reasoning tool, but all critical decisions (solvability verification, routing classification, iteration caps, budget enforcement) are handled by deterministic code. The distinction between "AI as tool-inside-the-system" and "AI as developer-of-the-system" is maintained throughout.

Appendix A: Repository Structure

PuzzleForge/ +-- README.md +-- AI_USAGE.md	# Project overview and setup instructions # AI tool usage disclosure
--	---

```

+-- requirements.txt          # Python dependencies
+-- .env.example             # Environment variable template
+-- docs/
|   \-- phase2_report.md     # This report
+-- src/
|   +-- models.py            # Pydantic data models (10 schemas)
|   +-- state.py             # LangGraph TypedDict state schema
|   +-- config.py            # Central configuration and thresholds
|   +-- main.py              # CLI entry point
|   +-- app.py               # Streamlit dashboard
|   +-- agents/
|   |   +-- game_designer.py # Game Designer agent (LLM, temp 0.8)
|   |   +-- level_designer.py # Level Designer agent (LLM, temp 0.7)
|   |   +-- developer.py     # Developer: translate + apply patches
|   |   +-- qa_tester.py     # QA Tester: solver + difficulty + diversity
|   |   \-- debugger.py      # Debugger agent (LLM, temp 0.3)
|   +-- solver/
|   |   \-- sokoban_solver.py # BFS solver with deadlock detection
|   +-- engine/
|   |   +-- template_engine.py # GameConfig -> playable HTML
|   |   \-- templates/
|   |       \-- sokoban.html  # HTML/JS game template
|   +-- orchestrator/
|   |   +-- graph.py          # LangGraph StateGraph definition
|   |   \-- routing.py        # Formal routing criteria (10 rules)
|   \-- utils/
|       +-- trace_logger.py   # Trace entry creation and formatting
|       \-- llm.py            # Centralized LLM call wrapper
+-- eval/
|   +-- test_cases.csv        # 12 test scenarios
|   +-- evaluation_plan.md    # Full evaluation methodology
|   +-- failure_log.md        # Documented failures and fixes
|   \-- version_notes.md      # Version history and known limitations
+-- outputs/
|   +-- sample_traces/
|   |   +-- sample_interaction_trace.json # Complete 9-step trace
|   |   \-- sample_game_config.json      # Pre-built 5-level game config
|   \-- sample_games/
\-- phase_submissions/
    +-- phase1/
    \-- phase2/

```

Appendix B: How to Run

```

# 1. Install dependencies
pip install -r requirements.txt

# 2. Set up environment
cp .env.example .env
# Edit .env with your OpenAI API key

# 3. Run demo (no API key needed)
python -m src.main --demo

# 4. Run with live API
python -m src.main "A push-block puzzle about a robot in a warehouse"

# 5. Run Streamlit dashboard
streamlit run src/app.py

```